Technische Hochschule Rosenheim

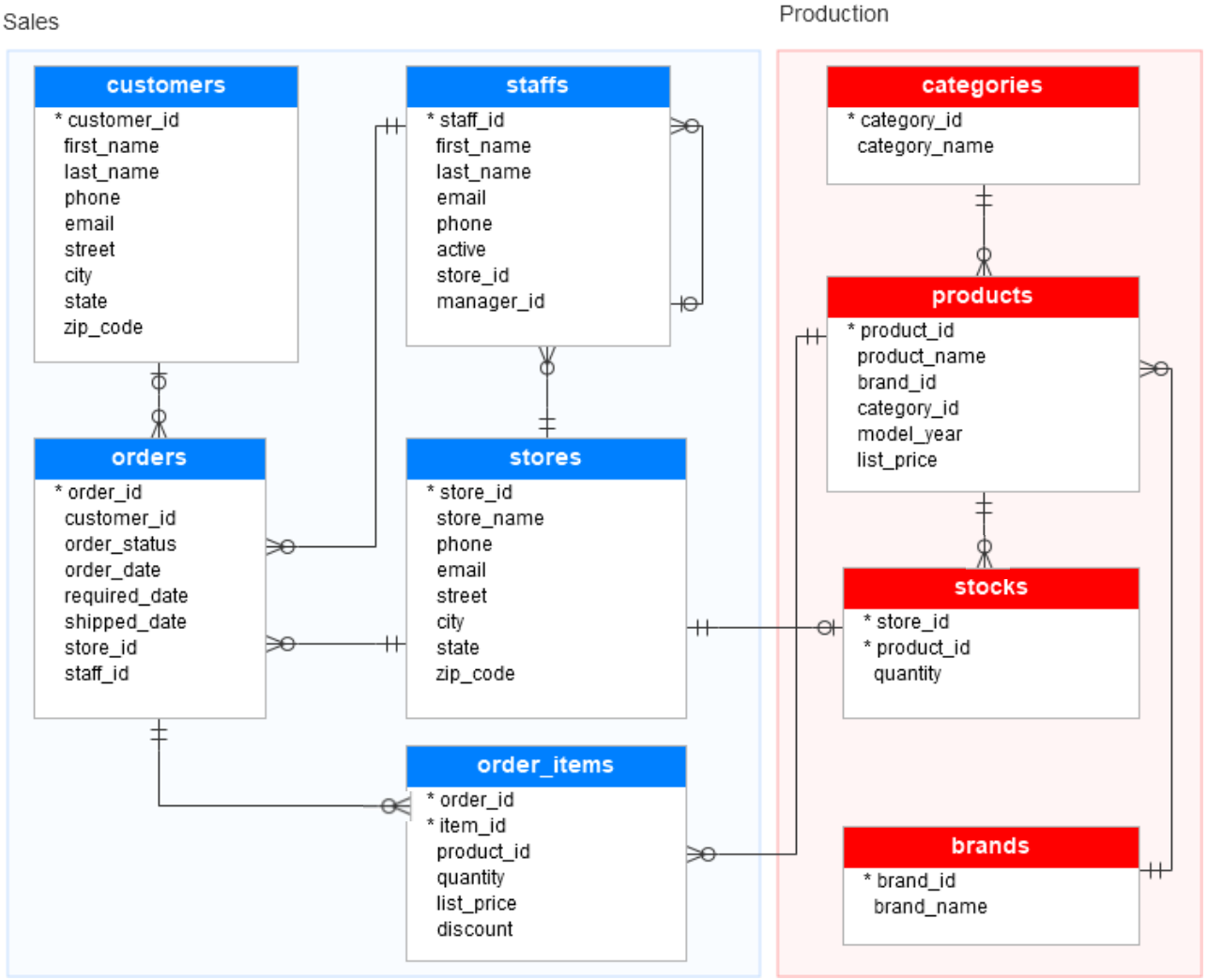
Chapter 7 – Advanced SQL

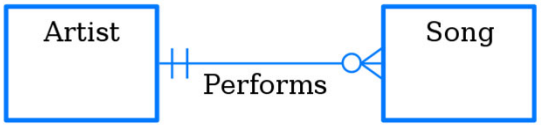
Part 1: Aggregates, Outer Joins, Top-k and Sorting

Databases lectures

Dr Kai Höfig

ER Diagram of the sample database





Crowsfoot notation is better for technical documentation, since it is more compact.

https://www.sqlservertutorial.net/sql-server-sample-database/

SQL core

- **`select`** projection list
  arithmetic operations & aggregate functions
- **`from`** relations to be used, possible renaming
- **`where`** selection conditions, join conditions
  nested queries (once again an SFW block)
- **`group by`** grouping for aggregate functions
- **`having`** selection conditions for groups
- **`order by`** output order
- Calculation sequence:
  `from`, **`where`**, **`group by`**, **`having`**, **`select`**, **`order by`**

Aggregate functions and grouping

- Aggregate functions calculate new values for an entire column, such as the sum or the average of the values in a column

- Examples:
  - Determination of the average price of all items or the total sales of all products sold
  - If grouping is also utilised: calculation of functions per group, e.g. the average price per product group or the total sales per customer

- Simple example: calculating the total number of customers

```
select COUNT(*) as 'Number of customers' from customers
```

```
Number of customers
1445
```

Aggregate functions in TSQL

- `count`: calculates the number of values in a column or alternatively (in the special case `count(*)`) the number of tuples in a relation
- `sum`: calculates the sum of the values in a column (only for numerical value ranges)
- `avg`: calculates the arithmetic mean of the values in a column (only for numerical value ranges)
- `max` or `min`: calculate the largest or smallest value in a column
- `stdev` or `var`: standard deviation or variance
- Arguments of an aggregate function
  - an attribute of the relation specified by the `from` clause,
  - a valid scalar expression or
  - in the case of the `count` function, also the `*` symbol

Aggregate functions in standard SQL (1)

- Before the argument (except in the case of `count(*)`) optionally also the keywords `distinct` or `all`
  - `distinct`: before using the aggregate function, the duplicate values are eliminated from the set of values to which the function is applied
  - `all`: duplicates are included in the calculation (default setting)
  - null values are eliminated from the set of values before the function is applied
    Exception: `count(*)`

Aggregate functions – some examples

- Number of different states in which the customers live:

```
select COUNT(distinct state) from customers
```

- Maximal delivery days:

```
select max(datediff(day,order_date,required_date))
from   orders
```

- Number of customers with a phone number:

```
select COUNT(*) from customers where phone is not null;

select COUNT(phone) from customers
```

Null values are eliminated!

Aggregate functions in the `where` or `from` clause

- Aggregate functions only return one value can be used in constant selections of the `where` clause, e.g. products that cost more than average.

```
select  product_name
from    products
where   list_price > (
                select AVG(list_price)
                from products)
```

- Or they can be part of the from clause to e.g. calculate the price deviation from the average price

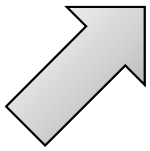
```
select  product_name, list_price-avg_price.price
from    products, (
select AVG(list_price) as price
from products) as avg_price
```

Grouping using `group by`

- Grouping using `group by` allows the summarising of tuples
- Example: Number of products per brand

```
select      brand_id,
            count (product_name)
from        products
group by    brand_id
```

brand_id	count
4	3
5	1



brand_id	product_name
4	Pure Cycles Vine 8-Speed - 2016
4	Pure Cycles Western 3-Speed - Women's - 2015/2016
4	Pure Cycles William 3-Speed - 2016
5	Ritchey Timberwolf Frameset - 2016

Grouping: schematic sequence (1)

- Relation

REL

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	4	1
5	4	4	3
…	…	…	…

Calculation sequence:
from, **where**, **group by**, **having**, **select**, **order by**

- Query:

```
select     A, sum(D) as D_TOTAL
from       REL
where      A<=4
group by   A, B
having     sum(D)<10 and max(C)=4
```

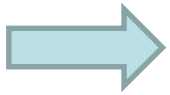
Grouping: schematic sequence (2)

Grouping: Step 1

- **`from`** and **`where`** **`(from`** `REL` **`where`** `A<=4)`

REL

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	4	1
5	4	4	3
…	…	…	…



Internal table in the DBMS, not visible externally!

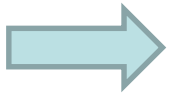
A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	4	1

Grouping: schematic sequence (3)

Grouping: Step 2

- **`group by`** `A, B`

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	4	1



A	B	N	
		C	D
1	2	3	4
		4	5
2	3	3	4
3	3	4	5
		6	7
4	3	4	1

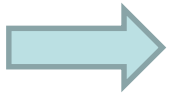
Internal tables in the DBMS, not visible externally!

Grouping: schematic sequence (4)

Grouping: Step 3

- **`having sum(D)<10 and max(C)=4`**

A	B	N	
		C	D
1	2	3	4
		4	5
2	3	3	4
3	3	4	5
		6	7
4	3	4	1



A	B	N	
		C	D
1	2	3	4
		4	5
4	3	4	1

Internal tables in the DBMS, not visible externally!

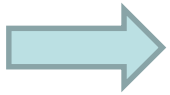
Grouping: schematic sequence (5)

Grouping: Step 4

- **`select`** A, **`sum`**(D) **`as`** D_TOTAL

A	B	N	
		C	D
1	2	3	4
		4	5
4	3	4	1

Internal table in the DBMS, not visible externally!



A	D_TOTAL
1	9
4	1

Results table

Selection of groups using `having`

- Selection of the grouping using **`having`** is possible
  - not possible in **`where`** due to the calculation sequence. Where is executed before the result of aggregate functions exist.

- Example: customers with more than one order

```
select          first_name,
                last_name,
                COUNT(*) as orders
from            customers,
                orders
where           orders.customer_id=customers.customer_id
group by        first_name, last_name
having          COUNT(*)>1
```

Exercise: grouping

- Names of the customers and value of their orders in the year 2017

Exercise: grouping

- Names of the customers and value of their orders in the year 2017

```
select  customers.first_name,
        customers.last_name,
        orders.order_date,
        SUM(order_items.list_price*order_items.quantity)
from    orders
        join customers on (orders.customer_id=customers.customer_id)
        join order_items on (order_items.order_id=orders.order_id)
where   datediff(day,cast('2017-01-01' as date),
                 orders.order_date) between 0 and 365
group by customers.first_name, customers.last_name,orders.order_date
```

Exercise: aggregate functions and nesting

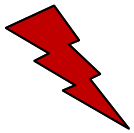
- Names of the customers and **average** value of their orders over all times

```
select  order_sums.first_name,
        order_sums.last_name,
        AVG(order_sums.value)
from    (selectcustomers.first_name,
        customers.last_name,
        orders.order_date,
        SUM(order_items.list_price*order_items.quantity) as value
        from orders
        join customers on (orders.customer_id=customers.customer_id)
        join order_items on (order_items.order_id=orders.order_id)
        group by customers.first_name,
        customers.last_name,orders.order_date) as order_sums
group by order_sums.first_name, order_sums.last_name
```

Aggregate functions - nesting

- Nesting of aggregate functions is not permitted.

```
select SUM(avg(x)) from y
```



- Instead, a subquery has to be used, which seems logical, if you execute the query step by step. (See example for nested aggregate functions)

Attributes for aggregation in `select` or `having`

- Permitted attributes after `select` when grouping of relation with schema R
  - Grouping attributes
  - Aggregations of non-grouping attributes
- Permitted attributes after `having`
  - Grouping attributes
  - Aggregations of non-grouping attributes
- When using aggregate functions, the attributes in select or having appear either in the group by or the aggregate function.

```
select          SUM(a), SUM(b), SUM(c), d, e, f
from            Z
group by        d,e,f
having          d=1 or SUM(a)=2
```

Equi Join or natural join

- Natural join and equi join in TSQL with the same syntax. Mostly used for key/foreign key relationships

- Examples:

```
select * from stores join staffs on (stores.store_id=staffs.store_id)
-- same as
select * from  staffs join stores on (stores.store_id=staffs.store_id)
-- same as
select * from  staffs,stores where stores.store_id=staffs.store_id
```

Theta Join

- Theta join is a join with a different join condition than equality. Same syntax as equi or natural join.

- Example:

```
select  store_name, product_id
from    stores join stocks on
        (stores.store_id=stocks.store_id and stocks.quantity<=0)
```

Semi Join

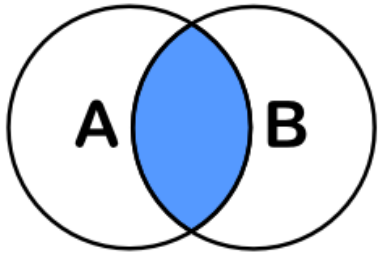
- In a semi-join, only attributes of one operand appear in the result. Same syntax as equi, natural or theta join.
  - Purpose: elimination of dangling tuples
  - Left semi join: only attributes of the left operand appear in the result
  - Right semi join: only attributes of the right operand appear in the result
- Example of a left semi join: all customers that did place an order

```
select distinct customers.*
from customers join orders on
        (customers.customer_id=orders.customer_id)
```

Inner join – the intersection

- The inner join can be used to extract the intersecting set of tuples that share for example a key/foreign key relationship (we called this a *natural join*).
- Example for products and stock. Contains no products that have no stock and no stocks that are not related to a product.

How do we find these?



SELECT <auswahl>
FROM tabelleA A
INNER JOIN tabelleB B
ON A.key = B.key

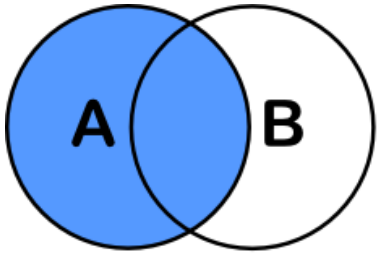
```
select  products.product_name,
        stocks.quantity
from    products join stocks on
        (products.product_id=stocks.product_id)
```

Left join – keep all tuples on the left, add data from the right

- When we want to keep all tuples from a relation but want to add data from another table e.g. by using a key/foreign key relationship, we can use the left join to add that data.

- Example: provide all customers and their orders if there are any.

```
select email,order_id
from customers
        left join orders on
        (customers.customer_id=orders.customer_id)
```



SELECT <auswahl>
FROM tabelleA A
LEFT JOIN tabelleB B
ON A.key = B.key

email	order_id
lezlie.lamb@gmail.com	558
ivette.estes@gmail.com	616
ester.acevedo@gmail.com	1424
jj@aol.com	NULL
miber@hotmail.com	NULL
wanita.douvenport@hotmail.com	NULL

This is basically the only outer join you need to remember

Exercise: Left join

- Provide all customers with their total sum of order values

Exercise: Left join

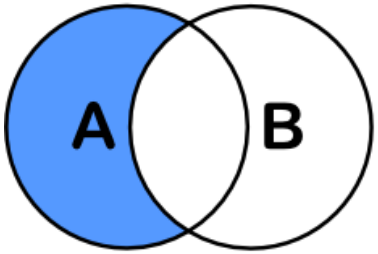
- Provide all customers with their total sum of order values

```
select  customers.first_name,
        customers.last_name,
        isnull(SUM(    order_items.list_price *
                       order_items.discount *
                       order_items.quantity),0) as value
from    customers
left join orders on           (customers.customer_id=orders.customer_id)
left join order_items on      (orders.order_id=order_items.order_id)
left join products on         (order_items.product_id=products.product_id)

group by customers.first_name, customers.last_name
order by value
```

Left join – only the complement

- An easy way to get those “special” customers, that did not order anything.



SELECT <auswahl>
FROM tabelleA A
LEFT JOIN tabelleB B
ON A.key = B.key
WHERE B.key IS NULL

```
select email,order_id
from customers
left join orders on
(customers.customer_id=orders.customer_id)
where orders.order_id is null
```

key of `orders`!
If A left join B, the key of B!

Right join – the left join upside down

```
select email,order_id
from  orders
        right join customers on
        (customers.customer_id=orders.customer_id)
where order_id is null
```

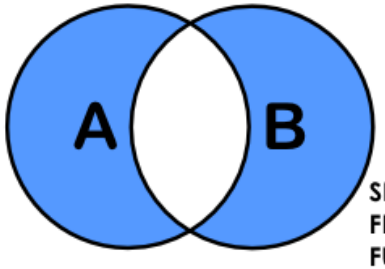
- Is the same as

```
select email,order_id
from customers
        left join orders on
        (customers.customer_id=orders.customer_id)
where orders.customer_id is null
```

Full Outer Join – rarely used in practice

- The full outer join is rarely used in practice, since it combines all rows from multiple tables using a join condition where applicable. It is most used to detect inconsistencies or to understand data.

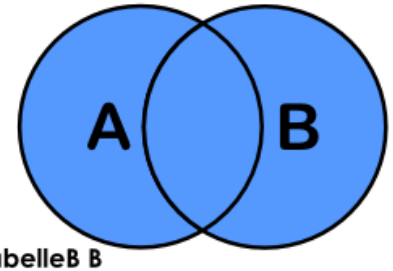
- Example: all customers that have no order and all orders with no customer



SELECT <auswahl>
FROM tabelleA A
FULL OUTER JOIN tabelleB B
ON A.key = B.key
WHERE A.key IS NULL
OR B.key IS NULL

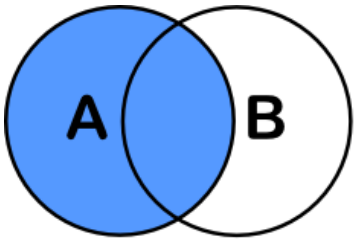
```
select  *
from    customers
full    outer join orders on
                (customers.customer_id=orders.customer_id)
where   orders.order_id is null or
        customers.customer_id is null
```

Without the where part: all tuples

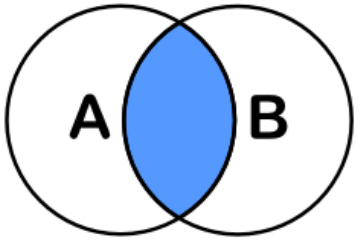


SELECT <auswahl>
FROM tabelleA A
FULL OUTER JOIN tabelleB B
ON A.key = B.key

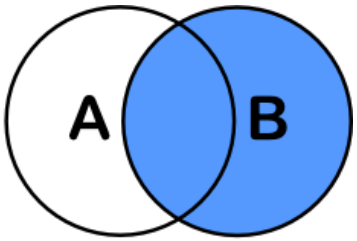
Outer joins – overview



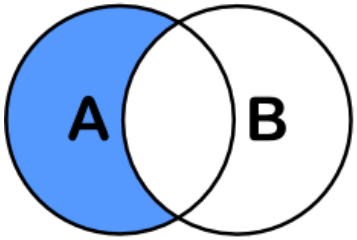

```
SELECT <auswahl>
FROM tabelleA A
LEFT JOIN tabelleB B
ON A.key = B.key
```



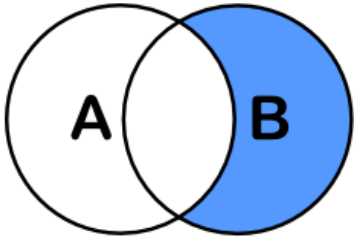

```
SELECT <auswahl>
FROM tabelleA A
INNER JOIN tabelleB B
ON A.key = B.key
```



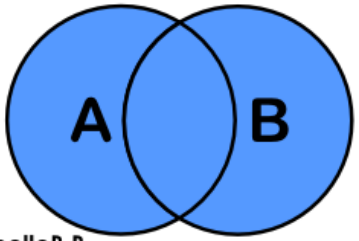

```
SELECT <auswahl>
FROM tabelleA A
RIGHT JOIN tabelleB B
ON A.key = B.key
```



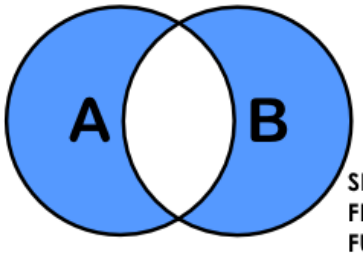

```
SELECT <auswahl>
FROM tabelleA A
LEFT JOIN tabelleB B
ON A.key = B.key
WHERE B.key IS NULL
```




```
SELECT <auswahl>
FROM tabelleA A
RIGHT JOIN tabelleB B
ON A.key = B.key
WHERE A.key IS NULL
```




```
SELECT <auswahl>
FROM tabelleA A
FULL OUTER JOIN tabelleB B
ON A.key = B.key
```




```
SELECT <auswahl>
FROM tabelleA A
FULL OUTER JOIN tabelleB B
ON A.key = B.key
WHERE A.key IS NULL
OR B.key IS NULL
```

Outer joins - replacement with union

- Outer joins are practical, but can also be replaced by union
- Example: left join

```
select email,order_id
from customers
       left join orders on
       (customers.customer_id=orders.customer_id)
```

same as

```
select  email,order_id
from    customers
join    orders on (customers.customer_id=orders.customer_id)
                union
select  email, null
from    customers
where   customers.customer_id not in
       (select customer_id from orders where customer_id is not null)
```

Sorting with `order by` (1)

- Notation

```
order by <attr_list>
```

- Example

```
select * from orders
order by       order_date desc,
               order_status asc,
               customer_id asc
```

- Sorting in ascending (**`asc`**) or descending (**`desc`**) order
- **`asc`** is default
- Sorting as the last operation of a query
  → Sorting attribute must occur in the **`select`** clause

Sorting with `order by` (2)

- Sorting also possible with calculated attributes (aggregates) as a sorting criterion

- Example

```
select          COUNT(*), customer_id
from            orders
group by        customer_id
order by        COUNT(*) desc
```

```
select          COUNT(*) as x, customer_id
from            orders
group by        customer_id
order by        x desc
```

Here, labels can be addresses in order by, also in group by, but **not** in where or having

Top-k clause

- Using top-k, a fraction of a query is kept in a result according to a ranking function.
- Example: latest three orders

```
select top(3) * from orders order by order_date desc
```

- Example: best three customers

```
select top(3)
       SUM(order_items.list_price*order_items.quantity) as value,
       orders.customer_id
from   orders join order_items on (orders.order_id=order_items.order_id)
group by orders.customer_id
order by value desc
```

Top-k queries without Top-k

```
select top(3) *
from orders
order by order_date
```

- Example: determine the 3 latest orders.
- Solution:

```
select      COUNT(*),
            o1.customer_id,
            o1.order_date
from        orders as o1,
            orders as o2
where       o1.order_date<o2.order_date
group by    o1.customer_id, o1.order_date
having      COUNT(*) <= 3
order by    COUNT(*) asc
```

- Result

	customer_id	order_date
1	135	2018-11-28
2	1	2018-11-18
3	3	2018-10-21

Sorting: Top-k queries (2)

- **Top-k query**: returns the best k elements with respect to a ranking function.
- **Design pattern:**
  - **Step 1:** Assignment of the required data sets to be able to calculate the ranking function
  - **Step 2:** Grouping according to the elements, calculation of the ranking
  - **Step 3:** Limiting to rankings ≤ k
  - **Step 4:** Sorting by rank
- Example: determining the k = 3 latest orders
  - Step 1: Assignment of all orders that are older
  - Step 2: Grouping according to the date and customer id
  - Step 3: Limiting to rankings ≤ 3
  - Step 4: Sorting by rank

Top-k Implemented in all important DBMS

Handling null values (1)

- Special value in SQL: **`null`**
  - Meaning: unknown or not applicable or not present (depending on the application)
- Test for **`null`** value:
  - *`attr`* **`is null`** returns **`true`**, if *`attr`* is **`null`**
  - *`attr`* **`is not null`** returns **`false`**, if *`attr`* is **`null`**
  - Example:

```
select * from orders where customer_id is null
```

- Terms: result is **`null`** whenever a null value is included in the calculation
  - Exception: aggregate functions: null values eliminated before the function is applied
  - Exception to the exceptions: null values are included in the case of **`count`**(*)
- Comparisons with null value: result in truth (Boolean) value **`unknown`**
  - There are thus 3 possible values for Boolean expressions: `true`, `false` and `unknown`

Handling null values (2)

- Boolean expressions are thus based on trinary logic
- Logical tables for trinary logic

AND	true	unknown	false
true	true	unknown	false
unknown	unknown	unknown	false
false	false	false	false

NOT	
true	false
unknown	unknown
false	true

OR	true	unknown	false
true	true	true	true
unknown	true	unknown	unknown
false	true	unknown	false